Anza RTS Alloc

Security Assessment

May 5th, 2026 — Prepared by OtterSec

Alpha Toure shxdow@osec.io

Thiago Tavares thitav@osec.io

Table of Contents

01 — Executive Summary

Overview

Anza engaged OtterSec to assess the `rts-alloc` library. This assessment was conducted between April 12th and April 18th, 2026. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 3 findings throughout this audit engagement.

In particular, we identified a vulnerability where file mapping leaks a kernel file-mapping handle if CreateFileMappingW succeeds, but MapViewOfFile fails (OS-ARA-ADV-00). Additionally, as allocation requests of zero size are not rejected, a zero-byte request still consumes a real slot from the smallest size class, wasting allocator capacity (OS-ARA-ADV-01).

We also suggested rephrasing the safety preconditions when freeing an offset to explicitly require an offset that exactly corresponds to a live allocation returned by the allocator (OS-ARA-SUG-00).

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/anza-xyz/rts-alloc. This audit was performed against commit f4634ad.

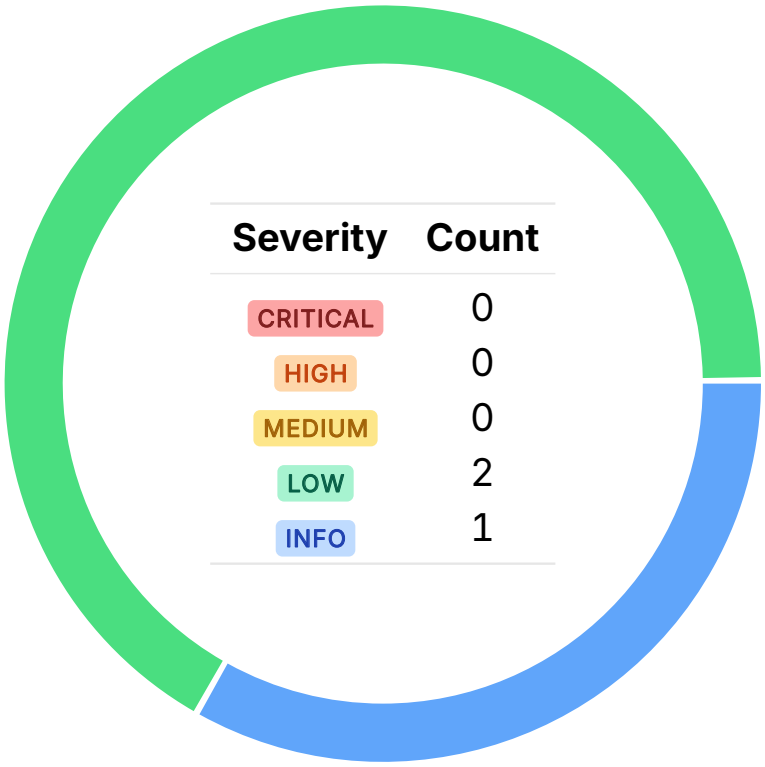
A brief description of the program is as follows:

Name	Description
rts-alloc	A lock-free shared-memory slab allocator for small objects, designed so multiple workers can allocate quickly from their own owned slabs with minimal contention. It supports cross-worker frees via deferred remote free lists and returns fully unused slabs back to a global pool.

03 — Findings

Overall, we reported 3 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	0
MEDIUM	0
LOW	2
INFO	1

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-ARA-ADV-00	LOW	RESOLVED	On Windows, `map_file` leaks a kernel file-mapping handle if `CreateFileMappingW` succeeds but `MapViewOfFile` fails, because it returns early without calling `CloseHandle(mapping)`.
OS-ARA-ADV-01	LOW	RESOLVED	`allocate(0)` is not rejected, so a zero-byte request still consumes a real slot from the smallest size class, wasting allocator capacity.

`map_file` on Windows Leaks Kernel Handles LOW OS-ARA-ADV-00

Description

On Windows, `memory_map::map_file` creates a file-mapping kernel object with `CreateFileMappingW`, which returns a mapping handle that must be closed. On the success path, the code correctly calls `CloseHandle(mapping)` after `MapViewOfFile`, but on the failure path, where `MapViewOfFile` returns a null view, it returns early without closing that handle. As a result, the process leaks a kernel handle each time this error path is hit.

>_ *src/memory_map.rs* RUST

```rust
#[cfg(windows)]
pub(crate) fn map_file(file: &File, size: usize) -> Result<*mut c_void, Error> {
    [...]
    let mapping = unsafe {
        CreateFileMappingW([...])
    };
    [...]

    let mmap = unsafe { MapViewOfFile(mapping, FILE_MAP_ALL_ACCESS, 0, 0, size) };

    if mmap.Value.is_null() {
        return Err(Error::MmapError(std::io::Error::last_os_error()));
    }

    unsafe {
        CloseHandle(mapping);
    }

    Ok(mmap.Value.cast())
}
```

Remediation

Call `CloseHandle(mapping)` before returning the `MmapError` when `MapViewOfFile` fails.

Patch

Resolved in PR#66.

Zero-Size Allocation Consumes Real Slot LOW OS-ARA-ADV-01

Description

`Allocator::allocate` delegates size bucketing to `size_class_index`, which does not reject `size == 0`. As a result, a zero-byte request is still mapped into the allocator's smallest size class and proceeds through the normal slab-allocation path, ultimately returning a real pointer backed by a normal slot, as the call to `size_class_index_unchecked(0)` returns 1 in `size_class_index` (`Some(unsafe size_class_index_unchecked(size))`). Thus, a zero-byte request is mapped to the allocator's smallest bucket and consumes a real slot, effectively burning 256 bytes of slab capacity. This is a correctness footgun because callers may reasonably expect zero-sized allocations to fail or to be free.

>_ *src/size_classes.rs* RUST

```rust
pub fn size_class_index(size: u32) -> Option<usize> {
    if size > MAX_SIZE {
        return None;
    }
    // SAFETY: size <= MAX_SIZE.
    Some(unsafe { size_class_index_unchecked(size) })
}

#[inline]
pub unsafe fn size_class_index_unchecked(size: u32) -> usize {
    size.next_power_of_two()
        .trailing_zeros()
        .saturating_sub(BASE_SHIFT) as usize
}
```

Remediation

Reject allocation requests of `size == 0`.

Patch

Resolved in PR#67.

05 — General Findings

Here, we present a discussion of general findings identified during our audit. While these findings do not pose an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-ARA-SUG-00	The safety documentation in `free_offset` is too vague, as not every in-range allocator offset is valid to free.

Missing Safety Precondition OS-ARA-SUG-00

Description

The current safety precondition for `allocator::free_offset` is underspecified because *"a valid offset within the memory allocated by this allocator"* may be read as any in-range byte offset into the mapped region. In reality, `free_offset` is only safe for offsets that identify the start of a live allocation slot, such as values previously produced by `offset(ptr)` for a pointer returned by this allocator.

>_ *src/allocator.rs* RUST

```rust
/// Free a block of memory previously allocated by this allocator.
///
/// # Safety
/// - The `offset` must be a valid offset within the memory allocated by this allocator.
/// - The `offset` must not have been freed before.
pub unsafe fn free_offset(&self, offset: usize) {
    let allocation_indexes = self.find_allocation_indexes(offset);
    // SAFETY: The allocation indexes are guaranteed to be valid by the caller.
    unsafe {
        self.base
            .remote_free_list(allocation_indexes.slab_index)
            .push(allocation_indexes.index_within_slab);
    }
}
```

Remediation

Rephrase the safety preconditions `free_offset` to explicitly state that valid offsets within allocated memory are limited to those pointing to blocks returned by `offset(ptr)`.

Patch

Resolved in PR#68.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.